

# Searching in an array

CS10003 PROGRAMMING AND DATA STRUCTURES



INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

# Passing an array to a function as argument

We want to pass an integer array to a function as argument. Let us say **the function will simply add the elements in the array and return the sum.**

Inside main( ):

```
int arr[100], n, j;  
printf("Enter number of elements: ");  
scanf("%d", &n);  
for( j = 0; j < n; j++ )  
    scanf( "%d", &arr[j] );
```

Now we want to send arr as argument to a function sumArr( )

# Passing an array to a function as argument

We want to pass an integer array to a function as argument.

Let us say **the function will simply add the elements in the array and return the sum.**

Attempt 1:

```
int sumArr( int a[ ] )  
{  
    int sum = 0, j;  
    // run a loop and add the elements of array a into sum  
}
```

**Problem: how many iterations should the loop inside the function have?**

# Passing an array to a function as argument

**Problem:** how many iterations should the loop inside the function have?

**Solution:** the function needs to know till where in the array there are valid elements

Attempt 2:

```
int sumArr( int a[ ], int limit )  
{  
    int sum = 0, j;  
    // run a loop and add the elements of array a into sum  
    for( j = 0; j < limit; j++ )    sum += a[ j ];  
    return sum;  
}
```

# Passing an array to a function as argument

```
int sumArr( int a[ ], int limit ){
    int sum = 0, j;
    for( j = 0; j < limit; j++ )    sum += a[ j ];
    return sum;
}

int main(){
    int arr[100], n, j;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    for( j = 0; j < n; j++ )    scanf( "%d", &arr[j] );
    total = sumArr( arr, n );
}
```



**Search**

# Searching

Check if a given element (called **key**) occurs in the array.

- Example: array of student records; **rollno** can be the key.

Two methods to be discussed:

- If the array elements are unsorted.
  - **Linear search**
- If the array elements are sorted.
  - **Binary search**

# Basic Concept of Linear Search

Basic idea:

- Start at the beginning of the array.
- Inspect elements one by one to see if it matches the key.
- If a match is found, return the array index where the match was found
- If no match is found, a special value is returned (e.g., -1)

# Linear Search

Function `linear_search` returns the array index where a match is found. It returns -1 if there is no match.

```
int linear_search (int a[], int size, int key)
{
    int pos = 0;
    while ((pos < size) && (a[pos] != key)) pos++;
    if (pos < size)
        return pos;          /* Return the position of match */
    return -1;               /* No match found */
}
```

# Time Complexity of Linear Search

A measure of **how many basic operations** an algorithm needs to perform before terminating.  
Example of basic operation: **match / compare two elements**

- If there are  $n$  elements in the array:
  - **Best case:**  
**match found in first element** (1 search operation)
  - **Worst case:**  
**no match found, or match found in the last element** ( $n$  search operations)
  - **Average case:**  $(n + 1) / 2$  search operations

# Binary Search

# Basic Concept

Binary search is applicable if the array is *sorted*.

## BASIC IDEA

- Look for the target in the middle.
- If you don't find it, you can ignore half of the array, and repeat the process with the other half.

In every step, we reduce the number of elements to search in by half.

# The Basic Strategy

What do we want?

- Find split between values larger and smaller than **key**:



- Situation while searching:
  - Initially  $L$  and  $R$  contains the indices of first and last elements.
- Look at the element at index  $[(L+R)/2]$ .
  - Move  $L$  or  $R$  to the middle depending on the outcome of test.

## The basic search iteration

```
int bin_search (int x[ ], int size, int key)
{
    int L, R, mid;
    _____;
    while ( _____ )
    {
        mid = (L + R) / 2;
        if (x[mid] <= key) L = mid;
        else R = mid;
    }
    _____;
}
```

If **key** appears in  $x[0..size-1]$ , return its location, pos such that  $x[pos]==key$ .

If not found, return -1

# Initialization and Return Value

```
int bin_search (int x[ ], int size, int key)
{
    int L, R, mid;
    L = -1; R = size;
    while ( L+1 != R )
    {
        mid = (L + R) / 2;
        if (x[mid] <= key) L = mid;
        else R = mid;
    }
    if (L >= 0 && x[L] == key) return L;
    else return -1;
}
```

If **key** appears in  $x[0..size-1]$ , return its location, pos such that  $x[pos]==key$ .

If not found, return -1

# Binary Search Examples

Sorted array

-17 -5 3 6 12 21 45 63 50

Output shown for key = 3

Trace :

bin\_search (x, 9, 3);

bin\_search (x, 9, 145);

bin\_search (x, 9, 45);

|        |      |           |
|--------|------|-----------|
| L= -1; | R=9; | x[4]=12;  |
| L= -1; | R=4; | x[1]= -5; |
| L= 1;  | R=4; | x[2]=3;   |
| L=2;   | R=4; | x[3]=6;   |
| L=2;   | R=3; | return L; |

We can modify the algorithm by checking equality with x[mid].

## A more efficient version

```
int bin_search_1 (int x[ ], int size, int key)
{
    int L, R, mid;
    L = 0; R = size-1;
    while ( L <= R )
    {
        mid = (L + R) / 2;
        if (x[mid] == key) return mid;
        if (x[mid] < key) L = mid+1;
        else R = mid-1;
    }
    return -1;
}
```

# Is it worth the trouble to sort the array before searching?

Suppose that the array **x** has 1000 elements.

## Ordinary search

– If **key** is a member of **x**, it would require 500 comparisons on the average.

## Binary search

- after 1st compare, left with 500 elements.
- after 2nd compare, left with 250 elements.
- After at most 10 steps, you are done.

# Time Complexity

If there are  $n$  elements in the array.

- Number of iterations required:

$$\log_2 n$$

For  $n = 64$  (say).

- Initially, list size = 64.
- After first compare, list size = 32.
- After second compare, list size = 16.
- After third compare, list size = 8.
- .....
- After sixth compare, list size = 1.

$2^k = n$ , where  $k$  is the  
number of steps.

$$\log_2 64 = 6$$

$$\log_2 1024 = 10$$

## Exercise

Write a recursive version of the Binary Search function.

# Recursive version of Binary search

// A recursive binary search function -- returns location of x in given array arr[l..r] is present, otherwise -1

```
int binarySearch( int arr[ ], int l, int r, int x )
```

```
{
```

```
    if ( r >= l ) {
```

```
        int mid = l + (r - l) / 2;
```

```
        if (arr[mid] == x) // If the element is present at the middle itself
```

```
            return mid;
```

```
        if (arr[mid] > x) // If element is smaller than mid, then it can only be present in left subarray
```

```
            return binarySearch(arr, l, mid - 1, x);
```

```
        // Else the element can only be present in right subarray
```

```
        return binarySearch(arr, mid + 1, r, x);
```

```
    }
```

```
    // We reach here when element is not present in array
```

```
    return -1;
```

```
}
```

```
int result = binarySearch(arr, 0, n - 1, x);
```

```
If (result == -1)
```

```
    printf("Key is not present in array")
```

```
else
```

```
    printf("Key is present at index %d", result);
```